CPU Temperature: 49.8 °C

Raspberry Pi 4 Monitoring

CPU Usage: 24.1%

Memory Usage: 43.7%

CPU Temperature: 50.1 °C

PRACTICAL-19

1. Aim

Design and implement an embedded access-control prototype that grants entry only when either a whitelisted RFID tag is detected or a legitimate fingerprint is verified, provides real-time visual feedback on an LCD, indicates status via an LED, and pulses a relay active-LOW) to unlock for a fixed interval. The system must log all events (time, method, esult) for auditability.

2. Prior Concepts

i) Concept of Python Programming

ii) Basics of RFID technology

iii) Basics of biometric authentication

vi) SPI communication protocol

v) UART communication protocol

vi) 12C communication protocol

vii) Relay interfacing basics

3. New Concepts

i) Development of a complete embedded access-control system.

ii) Integration of multiple authentication methods (RFID and Fingerprint).

iii) Event-driven access control system design.

4. Hardware Required

i) MFRC522 RFID Reader

ii) R307/AS608 Fingerprint Sensor

iii) 16x2 LCD (I2C)

iv) Relay Module (Active-LOW)

5. Procedure

Pin Connection

RFID (MFRC522) ---- Raspi:

MFRC522	Raspberry Pi
SDA	GPIO8
SCK	GPIO11
MOSI	GPIO10
MISO	GPIO9
RST	GPIO25
3.3V	3.3V
GND	GND

Fingerprint Sensor to Raspi:

Sensor	Raspberry Pi
VCC	5V
GND	GND
TX	GPIO15
RX	GPIO14

LCD 12C to Raspi:

LCD	Raspberry Pi
VCC	5V
GND	GND
SDA	GPIO2
SCL	GPIO3

LED to Raspi :

LED	Raspberry Pi
Anode	GPIO17 (via 2200)
Cathode	GND

Active Low Relay to Raspi:

Relay	Raspberry Pi
IN	GPIO27
VCC	5V
GND	GND

Note: Relay Logic

PD27= LOW -> Door Unlock

GP027 HIGH -> Door Locked

Input IMAGE

i) Install Python3 on Raspi.

Write the below code in command prompt:

sudo apt install idle3

Install Libraries

pip3 install mfrc522

pip3 install pyfingerprint

pip3 install RPLCD

pip3 install RPLCD-12c

pip3 install spidev

6. Implementation with Python Code

Code:

```
import time

import csv

from datetime import datetime

import RPI.GPIO as GPIO

from mfrc522 import SimpleMFRC522
```

```
from pyfingerprint.pyfingerprint import PyFingerprint

from RPLCD.i2c import CharLCD

# ==========================================

#CONFIGURATION

# ==========================================

LED_PIN = 17

RELAY_PIN = 27                    #Active-LOW relay

UNLOCK_TIME = 5                   #seconds

LOG_FILE = "access_log.csv"

# Replace with your RFID UID(s)

AUTHORIZED UIDS = [

123456789012,

987654321098

]

# ==========================================

#GPIO SETUP

# ==========================================

GPIO.setwarnings(False)

GPIO.setmode (GPIO.BCM)

GPIO.setup(LED_PIN, GPIO.OUT)
```

```
GPIO.setup(RELAY_PIN, GPIO.OUT)

GPIO.output(LED_PIN, GPIO.LOW)

#Relay OFF (locked)

GPIO.output(RELAY PIN, GPIO.HIGH)

# = = =
```

```
try:
fp = PyFingerprint(
'/dev/serialo',
57600,
0xFFFFFFFF
0x00000000
)
if not fp.verify Password():
raise ValueError(
'Fingerprint sensor password incorrect'
)
wept Exception as e:
print("Fingerprint initialization failed")
print(e)
Icd.clear()
icd.write_string("FP Init Error")
exit()
# = = =
```

```
def lcd_msg(line1="", line2=""):

lcd.clear()

lcd.write_string(str(line1)[:16])

if line 2:

lcd.crlf()

lcd.write_string(str(line2)[:16])

# = = = = = = = = = = = = = = = = = = = = = = = = = = = = = = = = = = = = = = = = = = = = = = = = =

#LOGGING

# = = = = = = = = = = = = = = = = = = = = = = = = = = = = = = = = = = = = = = = = = = = = = = = = =

def log_event(method, identifier, result):

timestamp = datetime.now().strftime(

"%Y-%m-%d %H:%M:%S"

)

with open(LOG_FILE, "a", newline="") as file:

writer = csv.writer(file)

writer.writerow([

timestamp,

method,

identifier,

result
```

```
])
#==========================================
#ACCESS FUNCTIONS
#==========================================
def unlock_door():
GPIO.output(RELAY_ output (RELAY_PIN, GPIO.LOW)
time.sleep(UNLOCK_TIME)
GPIO.output(RELAY_PIN, GPIO.HIGH)
def access_granted(method, identifier):
print(
f"[GRANTED] [method): (identifier)"
)
lcd_msg(
"ACCESS",
"GRANTED"
)
GPIO.output(
LED_PIN,
GPIO.HIGH
)
```

```
log event(
method,
identifier,
"GRANTED"
)
unlock_door()
GPIO.output
LED_PIN,
GPIO.LOW
)
time.sleep(1)
def access_denied(method, identifier):
print(
f"[DENIED] (method): (identifier)"
)
lcd_msg(
"ACCESS",
"DENIED"
)
log_event(
```

```
method,

identifier,

"DENIED"

)

time.sleep(2)

# ==========================================

# RFID CHECK

# ==========================================

def check_rfid():

try:

uid, text = reader.read_no_block()

If uld is None:

return False

print("RFID UID:", uid)

if uid in AUTHORIZED_UIDS:

Icd_msg(

"RFID VERIFIED",

"WELCOME"

)

access_granted(
```

```
"RFID",
Uid
)
else:
lcd_msg(
"RFID INVALID",
str(uid)
)
access_denied(
"RFID",
uid
)
return True
Except Exception as e:
print("RFID Error:", e)
return False
# ==========================================
#FINGERPRINT CHECK
# ==========================================
def check_fingerprint():
```

```
try:
if not fp.readImage():
return False
fp.convertImage(0x01)
result = fp.searchTemplate()
position = result[0]
if position >= 0:
lcd_msg(
"FINGER MATCH",
f"ID:{position}"
)
access_granted(
"FINGERPRINT",
Position
)
else:
Icd_msg(
"NO MATCH",
"UNKNOWN"
)
```

```
access_denied(

"FINGERPRINT",

"UNKNOWN"

)

return True

except Exception as e:

print("Fingerprint Error:", e)

return False

# ==========================================

#MAIN

# ==========================================

def create_log_file():

try:

with open(

LOG_FILE,

"x",

newline=""

) as file:

writer = csv.writer(file)

writer.writerow([
```

```
"Timestamp",
"Method",
"Identifier",
"Result"
])
except FileExists Error:
pass
def main():
create_log_file()
lcd_msg(
"ACCESS CONTROL",
"Ready"
)
print(
"System Ready..."
)
try:
Icd_msg(
"Scan Card or",
"Place Finger"
```

```
)
rfid_detected = check_rfid()
if rfid_detected:
continue
finger_detected = check_fingerprint()
if finger_detected:
continue
time.sleep(0.2)
except KeyboardInterrupt:
print("Exiting...")
finally:
lcd.clear()
GPIO.output(
RELAY_PIN,
GPIO.HIGH
)
GPIO.output(
LED_PIN,
GPIO.LOW
)
```

```
GPIO.cleanup()

# = = = = = = = = = = = = = = = = = = = = = = = = = = = = = = = = = = = = = = = = = = = = = = = = = = = =

if_name__ = = "__main__":

main()
```

7. Result :

PRACTICAL-20

1. Aim

install Windows IoT 10 core on Raspberry pi and run the demo program.

2. Prior Concepts

i) Basic computer architecture and operating systems.

ii) Windows operating system basics.

iii) Microprocessor and microcontroller fundamentals.

iv) Basic networking concepts (IP address, Ethernet, Wi-Fi).

3. New Concepts

i) Windows 10 IoT Core operating system.

ii) Deployment of Windows lot on Raspberry Pi.

iii) Using Windows IoT Dashboard.

iv) Universal Windows Platform (UWP) applications.

4. Procedure